

Análise do desempenho de uma aplicação de multiplicação de matrizes entre P-Core e E-Cores

JOÃO LUIZ GRAVE GROSS, Instituto de Informática da UFRGS, Brasil

[TODO]

Additional Key Words and Phrases: P-Core, E-Core, Intel

ACM Reference Format:

João Luiz Grave Gross. 2025. Análise do desempenho de uma aplicação de multiplicação de matrizes entre P-Core e E-Cores. 1, 1 (November 2025), 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introdução

[TODO] p-cores performance e-cores energia/econômico

2 Plataforma: Hardware e SO

A plataforma de execução consiste em uma estação de trabalho Dell OptiPlex 7020 Micro. As especificações de *hardware* incluem uma CPU Intel Core i7-14700T (14ª Geração, 20 cores, sendo 8 p-cores/16 threads e 12 e-cores/12 threads, cache de 33 MB, 1.3GHz a 5.0GHz), 16GB de RAM DDR5 5600MT/s (módulo único) e SSD de 512GB NVMe M.2 [1]. O ambiente de *software* baseia-se no sistema operacional Ubuntu 24.04 LTS [3].

O mini desktop em questão foi escolhido, pois o autor tem acesso irrestrito ao equipamento (24/7) e é usuário único, permitindo que os experimentos pudessem ser executados a qualquer tempo, sem a necessidade de compartilhamento de recursos. Adicionalmente, o equipamento está fisicamente instalado em um Data Center, onde a temperatura é mantida estável entre 17°C e 21°C, mitigando oscilações térmicas que poderiam interferir nos resultados.

3 Aplicação e Características dos Scripts

A multiplicação de matrizes foi selecionada como a carga de trabalho base para a avaliação de desempenho entre *p-cores* e *e-cores*. Esta escolha fundamenta-se na natureza estritamente *CPU-bound* do problema e em sua alta capacidade de paralelização, características que permitem mensurar o desempenho computacional do processador e distribuir um conjunto de tarefas entre os múltiplos núcleos.

Foram desenvolvidos 3 scripts para computar a multiplicação de matrizes. O primeiro foi desenvolvido para Python 3.12.12 utilizando Multiprocessing, o segundo também para Python 3.12.12 utilizando Dask e, por fim, o terceiro foi desenvolvido para Python 3.14.0 com Threading, visto que esta última versão do python não possui mais o bloqueio de GIL.

A Foram desenvolvidas três implementações distintas de scripts para a computação da multiplicação de matrizes. As duas primeiras foram executadas sobre o interpretador *Python 3.12.12*: uma

Author's Contact Information: João Luiz Grave Gross, jlgross@inf.ufrgs.br, Instituto de Informática da UFRGS, Porto Alegre, Rio grande do Sul, Brasil.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2025/11-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

utilizando a biblioteca *multiprocessing* para paralelismo baseado em processos, e a outra empregando o *framework Dask* para computação paralela. A terceira implementação foi desenvolvida para o *Python 3.14.0* utilizando o módulo *threading*, visando explorar a remoção do *Global Interpreter Lock* (GIL), recurso nativo desta versão que permite paralelismo real entre *threads*.

8 threads/processos, sempre

- 3 scripts: características de cada um - coleta de temperatura ao término de cada execução - registro do tempo final de cada execução - cenários: p-core sem HT, p-core com HT, p-core+e-core, e-core only - governor: sempre performance

4 Cenários dos Testes

5 Estudos Preliminares e Validações

Para permitir que as execuções dos scripts ocorressem em *p-cores* e *e-cores* específicos, utilizou-se a **afinidade de CPU** a nível de sistema operacional com o comando `os.sched_setaffinity(pid, \{cpu_id\})`.

Explicar características de compartilhamento de memória para multiprocessing, dask e threading Herança de memória do pai, compartilhamento de memória, zero copy (thread), copy on write (multiprocessing) Impacto do GIL para versões Python < 3.14 Python 3.12 sem possibilidade de Threading por causa da trava do GIL Validação de uso de núcleos específicos com set affinity

6 Configurações Iniciais

[TODO] - pyenv e versões de python - variáveis de ambiente, BLAS, numpy uma thread - Comandos para execução do Jupyter notebook com variáveis de ambiente, seja python 3.12 ou Python 3.14

7 Projeto Experimental

- 3 scripts - 4 cenários - 5 tamanhos de matriz: máximo de 8000 para evitar problema de memória no script Dask - 30 repetições: para aproximação de uma distribuição gaussiana real e para aumentar intervalo de confiança

- adição de cargas de warmup (descartadas na análise)

- embaralhamento dos 600 experimentos + 8 cargas de warmup iniciais matrix_size 8000

8 Análise dos Resultados

8.0.1 Script 1.

9 Reprodutibilidade

- github: link - pilha de software: GUIX/NIX

10 Conclusões

11 Sectioning Commands

11.1 Subsection

This is a subsection.

11.1.1 Subsubsection. This is a subsubsection.

Paragraph. This is a paragraph.

Subparagraph This is a subparagraph.

Tabela 1. Frequency of Special Characters

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	For Swedish names
π	1 in 5	Common in math
\$	4 in 5	Used in business
Ψ_1^2	1 in 40,000	Unexplained usage

Tabela 2. Some Typical Commands

Command	A Number	Comments
<code>\author</code>	100	Author
<code>\table</code>	300	For tables
<code>\table*</code>	400	For wider tables

12 Tables

The “acmart” document class includes the “booktabs” package — <https://ctan.org/pkg/booktabs> — for preparing high-quality tables.

Immediately following this sentence is the point at which Table 1 is included in the input file; compare the placement of the table here with the table in the printed output of this document.

To set a wider table, which takes up the whole width of the page’s live area, use the environment **table*** to enclose the table’s contents and the table caption. As with a single-column table, this wide table will “float” to a location deemed more desirable. Immediately following this sentence is the point at which Table 2 is included in the input file; again, it is instructive to compare the placement of the table here with the table in the printed output of this document.

Always use `midrule` to separate table header rows from data rows, and use it only for this purpose. This enables assistive technologies to recognise table headers and support their users in navigating tables more easily.

13 Figures

The “figure” environment should be used for figures. One or more images can be placed within a figure. If your figure contains third-party material, you must clearly identify it as such, as shown in the example below.

Fig. 1. 1907 Franklin Model D roadster. Photograph by Harris & Ewing, Inc. [Public domain], via Wikimedia Commons. (<https://goo.gl/VLCRBB>).

Some examples. A paginated journal article [2].

Referências

[1] Dell. 2025. Desktop OptiPlex Micro. <https://www.dell.com/pt-br/shop/cty/pdp/spd/optiplex-7020-micro>.
[2] Michael Downes and Barbara Beeton. 2004. *The amsart, amsproc, and amsbook document classes*. American Mathematical Society. <http://www.ctan.org/pkg/amslatex>.
[3] Ubuntu. 2025. ubuntu 24.03 LTS. <https://ubuntu.com/download/desktop>.